Optimized Trajectory Design for Lunar Flyby Mission (MATLAB)

By: Noha Fathy Hassen

August 1, 2025

Table of Contents

INTRODUCTION **3**
1. What is orbit optimization? **4**
2. Approach Used & Results **4**
3. Program Parameters and Constants **5**
4. Trajectory Analysis and Mathematical Model 8
4.1. Initial Transfer Calculations 8
4.2. Optimization mathematical model 10
4.3. Lunar arrival Julian day calculation 10
4.4. State vector of the moon at arrival 10
4.5. RAAN & Dec from RAAN R 11
4.6. Initial state vector of the spacecraft 12
4.7. Motion Integration 13
4.8. Transformation from ECI to moon-fixed frame DCM 13
4.9. Spacecraft's Perilune distance determination 14
4.10. The history of the trajectory's binormal: 15
5. Trajectory Optimization function 16
5.1. Before Optimization 16
5.2. Applying Optimization 16
5.3. After Optimization 17
6. Results and Outputs 20
7. MATLAB Script 24
8. References 38

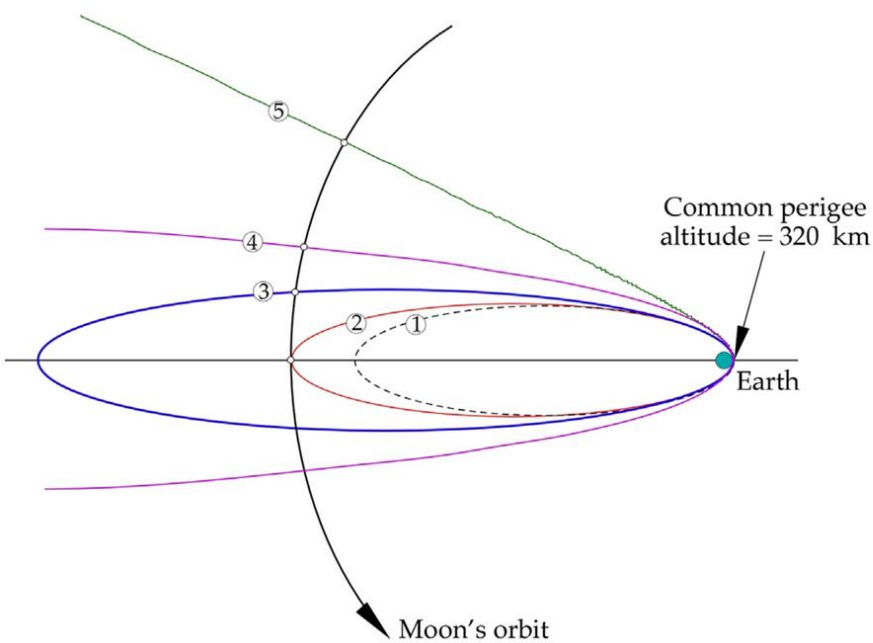
INTRODUCTION

This project simulates a lunar mission by analyzing various Earth-centered elliptical transfer orbits that aim to intersect the Moon's orbit. The method involves incrementally increasing the perigee speed (velocity at the closest point to Earth) to observe its effect on the spacecraft's trajectory and time of flight.

- Orbit 1 (10.8 km/s): Fails to reach the Moon's orbit.
- Orbit 2–4 (10.85–10.9 km/s): Reach and intersect the Moon's orbit, with improved flight times.
- Orbit 5 (11.2 km/s): Further reduces flight time but requires a much higher ΔV (fuel cost).

Each of these orbits shares a common perigee altitude of 320 km, which simplifies comparison.

However, reaching the Moon's orbit is not enough the spacecraft must arrive synchronously with the Moon's position to allow gravitational capture, landing, or orbit insertion. Thus, lunar gravity becomes a crucial factor, particularly as the spacecraft nears lunar distance, requiring a three-body dynamical model to predict the actual path under Earth and Moon gravities.



1. What is orbit optimization?

Orbit optimization in this project refers to the process of improving the spacecraft's transfer trajectory from Earth to the Moon to minimize fuel consumption, reduce ΔV (delta-v), and increase mission efficiency.

2. Approach Used & Results

I used a parameter called fac, which adjusts the shape of the transfer orbit. By applying an optimization algorithm (specifically fminbnd), then searched for the value of fac that leads to the lowest possible fuel usage while still achieving a successful lunar transfer.

The results showed that by optimizing this parameter:

- Fuel consumption was reduced by approximately 860 kg compared to the nominal (unoptimized) case.
- ΔV was slightly reduced, meaning less thrust was needed.
- Overall maneuver efficiency increased, making the trajectory more cost-effective and practical for mission planning.

This optimization step plays a crucial role in real-world space missions where fuel is limited, and every kilogram saved can be used for other mission-critical operations.

3. Program Parameters and Constants

m0	Initial mass of the spacecraft in Kg
g0	Standard Gravitational Acceleration
Isp	specific impulse of the propellant.
deg	Conversion factor, degrees to radians.
days	Conversion factor, days to seconds.
Re,Rm	Radii of earth and moon, respectively (km)
alpha0, dec0	initial right ascension and declination of spacecraft (deg)
gamma0	initial flight path angle (deg)
m_e, m_m	Masses of earth and moon, respectively (kg)
mu_e, mu_m	Gravitational parameters of earth and moon, respectively (km^3/s^2)
D	Semi major axis of moon's orbit (km)
RS	Radius of moon's sphere of influence (Km)
fac	ratio of spaccraft's initial speed to the escape speed.
ttt	predicted time to perilune (s)
tf	time at end of trajectory (s)
alt0	initial altitude of the trajectory (km)
z0	initial radius from earth's center to probe (km)
r0_mag	initial ECI position vector of probe (km)
speed_at_LEO	initial ECI speed of probe (km/s)
vesc	escape speed at r0 (km/s)
speed_TLI	Magnitude of probe's velocity at TLI.
delta_v	Delta v used for TLI.
efficiency	Percentage of initial mass used as fuel during the maneuver
fac_min	minimum value of fac range.
fac_max	Maximum value of fac range.
objective	Function to minimize fuel usage during optimization.
opt_efficiency	Fuel efficiency after optimization, as a percentage of initial mass.
Year, month, day, hour, minute, second	Date and time of spacecraft's lunar arrival.
UT	Universal time at lunar arrival.
t0	initial time on the trajectory (s)
jd0	julian date of lunar arrival
rm0, vm0	state vector of the moon at jd0 (km, km/s)

RA, Dec	right ascension and declination of the moon at jd0 (deg)
distance	
Hmoon, hmoon_mag	moon's angular momentum vector and magnitude at jd0 (km^2/s)
inclmoon	inclination of moon's orbit earth's equatorial plane (deg)
I,J,K	Unit vectors of ECI frame.
w0	unit vector normal to plane of translunar orbit at time t0
ur	radial unit vector to probe at time t0
uperp	transverse unit vector at time t0
vr_mag	initial radial speed of probe (km/s)
vperp_mag	initial transverse speed of probe (km/s)
v0	initial velocity vector of probe (km/s)
uv0	initial tangential unit vector
y0	initial state vector of the probe (km, km/s)
t	vector containing the times from t0 to tf at which the state vector is evaluated (s)
y	a matrix whose 6 columns contain the inertial position and velocity components evaluated at the times t(:) (km, km/s)
X,Y,Z	the probe's inertial position vector history
vX,vY, vZ	the probe's inertial velocity history
x,y,z	the probe's position vector history in the Moon-fixed frame
Xm,Ym, Zm	the Moon's inertial position vector history
vXm, vYm, vZm	the Moon's inertial velocity vector history
xm, ym, zm	the moon's inertial position vector history
dist_min	perilune of trajectory (km)
ti	the ith time of the set [t0,tf] (s)
jd	julian date of corresponding to ti (days)
rm,vm	the moon's state vector at time ti (km,km/s)
i_hat,j_hat,k_hat	unit vectors of the moon-fixed rotating frame at time ti
Q	DCM of transformation from ECI to moon-fixed frame at time ti
r_moon_fixed	probe's inertial position vector in moon fixed coordinates at time ti (km)
r	probe's inertial position vector at time ti (km)
rm_moon_fixed	Moon's inertial position vector in moon fixed coordinates at time ti (km)

distance_vector	position vector of probe relative to the moon at time ti (km)
current_distance	magnitude of dist_ (km)
rmTLI	Moon's position vector at TLI
RATLI, DecTLI	Moon's right ascension and declination at TKI (deg)
rm_perilune,vm_perilune	Moon's state vector when the probe is at perilune (km, km/s)
RA_at_perilune, Dec_at_perilune	Moon's RA and Dec at perilune arrival (deg)
v_atdmin	Probe's velocity vector at perilune (km/s)
rel_speed	Speed of probe relative to the Moon at perilune (km/s)
target_error	Distance between Moon's actual position at perilune arrival and its position after the predicted flight time, ttt (km).
rend	Position vector of end point of trajectory (km)
a_earth	acceleration of spacecraft due to earth (km/s^2)
a_moon	acceleration of spacecraft due to moon (km/s^2)
alt_end	Altitude of end point of trajectory (km)
ra_end, dec_end	Right ascension and declination of end point of trajectory (km)
atotal	a_earth_+ a_moon (km/s^2)
binormal	unit vector normal to the osculating plane
binormal_z	angle between inertial Z axis and the binormal (deg)
deltaV_total	Total delta_v for the mission
burn_duration	

4. Trajectory Analysis and Mathematical Model

To simulate or analyze the spacecraft's trajectory, we need:

- **Initial conditions:** The spacecraft's position and velocity (in low Earth orbit).

  Altitude: z=320km

  Right ascension: α=90°

  Declination: δ=15°

  Flight path angle: γ=40°

  Speed: v=10.8267km/s

  Spacecraft initial mass: 28801 Kg

- **Lunar position data rm(t):** Known from ephemeris function.
- **A numerical integrator:** Such as the Runge-Kutta method or MATLAB's ode45, to solve the system over time

We assume the spacecraft starts in a Low Earth Orbit (LEO) and performs a Trans-Lunar Injection (TLI) maneuver.

4.1. Initial Transfer Calculations

We get initial spacecraft's radius and speed from:

$$z_0 = alt0 + Re \tag{1}$$

Where Re is the earth radius and $alt0$ is the initial altitude of the spacecraft.

Then we calculate the initial speed at LEO from this equation:

$$speed_{at_{LEO}} = \sqrt{\frac{mu_e}{z0}}$$

We need to calculate the escape velocity from earth by this formula:

$$vesc = \sqrt{\frac{2 * mu_e}{z0}}$$

Then we get the required speed for TLI (transfer lunar insertion)

$$speed_{TLI} = fac * vesc \qquad (3)$$

We get now to calculate the delta v required for this maneuver:

$$delta_v = speed_{TLI} - speed_at_{LEO}$$

We assume that initial mass of the spacecraft is 28801 Kg, then calculate nominal fuel for the transfer through this formula:

$$fuel = \frac{\mathrm{m_0}}{\mathrm{e}\frac{\mathrm{delta_v}}{\mathrm{Isp} * \mathrm{g0}}}$$

Note that delta_v here into converted to m/s.

If we needed to determine the efficiency of the thrust, then we could apply this formula:

$$\text{Efficiency} = \frac{fuel}{m0} * 100$$

4.2. Optimization mathematical model

We set a range for fac to determine the best values to change speed through a built in function in MATLAB “fminbnd”, then we can reapply it in equation 3 and complete the process to get optimized delta_v and minimal fuel used and efficiency after optimization.

4.3. Lunar arrival Julian day calculation

We calculate the Julian date of lunar arrival through this formula:

$$J_0 = 367y - \mathrm{INT}\left\{\frac{7\left[y + \mathrm{INT}\left(\frac{m+9}{12}\right)\right]}{4}\right\} + \mathrm{INT}\left(\frac{275m}{9}\right) + d + 1{,}721{,}013.5$$

$$jd0 = j0 + \frac{UT}{24}$$

Where j0 is the Julian day number at 0h UT

4.4. State vector of the moon at arrival

lunar ephemeris table is used here to determine the position and velocity of the moon at a specific Julian day:

$$X_i = \sum_{j=1}^{7} a_{ij} \sin\left(b_{ij}t + c_{ij}\right) \quad (i = 1, 2, 3)$$

where X1= X, X2=Y, X3=Z, and t is the time in Julian centuries since J2000

$$t = \frac{JD - 2{,}451{,}545}{36{,}525} \text{ (centuries)}$$

and JD is the Julian date (in days). The components of the 3-by-7matrices[a], [b], and[c]are

$$[\mathbf{a}] = \begin{bmatrix} 383{,}000 & 31{,}500 & 10{,}600 & 6{,}200 & 3{,}200 & 2{,}300 & 800 \\ 351{,}000 & 28{,}900 & 13{,}700 & 9{,}700 & 5{,}700 & 2{,}900 & 2{,}100 \\ 153{,}200 & 31{,}500 & 12{,}500 & 4{,}200 & 2{,}500 & 3{,}000 & 1{,}800 \end{bmatrix} \text{ (km)}$$

$$[\mathbf{b}] = \begin{bmatrix} 8399.685 & 70.990 & 16728.377 & 1185.622 & 7143.070 & 15613.745 & 8467.263 \\ 8399.687 & 70.997 & 8433.466 & 16728.380 & 1185.667 & 7143.058 & 15613.755 \\ 8399.672 & 8433.464 & 70.996 & 16728.364 & 1185.645 & 104.881 & 8399.116 \end{bmatrix} \text{ (rad/century)}$$

$$[\mathbf{c}] = \begin{bmatrix} 5.381 & 6.169 & 1.453 & 0.481 & 5.017 & 0.857 & 1.010 \\ 3.811 & 4.596 & 4.766 & 6.165 & 5.164 & 0.300 & 5.565 \\ 3.807 & 1.629 & 4.595 & 6.162 & 5.167 & 2.555 & 6.248 \end{bmatrix} \text{ (rad)}$$

The moon's geocentric equatorial velocity components are found by simply differentiating position Eq with respect to time,

$$\dot{X}_i = \frac{1}{t_C} \sum_{j=1}^{7} a_{ij} b_{ij} \cos\left(b_{ij} t + c_{ij}\right) \quad (i = 1, 2, 3)$$

4.5. RAAN & Dec from RAAN R

We get the initial declination and right ascension of the moon from its position by applying this formula:

$$\text{RA} = \cos^{-1}\left(\frac{1}{\cos \text{Dec}}\right)$$

$$\text{Dec} = \sin^{-1}\left(\frac{\text{R_z}}{R_mag}\right)$$

Where is R_mag is the magnitude of position vector, and R_z is the third component of the vector. note that RAAN should be in range from 0to 360 degree so if its higher than this limit, you can subtract it from 360 till you get into the range.

Then, we calculate the inclination of the moon by getting the angular momentum first:

$$hmoon = rm0\ x\ vm0$$

$$inclmoon = \cos^{-1}\left(\frac{\text{hmoon_z}}{hmoon_mag}\right)$$

4.6. Initial state vector of the spacecraft

Since we have initial radius of spacecraft, RAAN and Dec so we can calculate the position vector from it by this equation:

$$r0 = r0_{mag} * \cos(alpha0) * \cos(dec0) * I + \sin(alpha0) * \cos(dec0) * J + \sin(dec0) * K$$

Where I, J, K is the initial unit vectors of ECI frame.

Both of the position vectors r0 and rm define the initial plane of the translunar trajectory, which means that the unit normal $\hat{w}$ to that orbital plane may be found by normalizing the cross product of r0 into rm,

$$\hat{\mathbf{w}} = \frac{\mathbf{r}_0 \times \mathbf{r}_m}{\|\mathbf{r}_0 \times \mathbf{r}_m\|}$$

The unit vector $\widehat{ur}$ in the direction of r0 is

$$\hat{\mathbf{u}}_r = \frac{\mathbf{r}_0}{r_0}$$

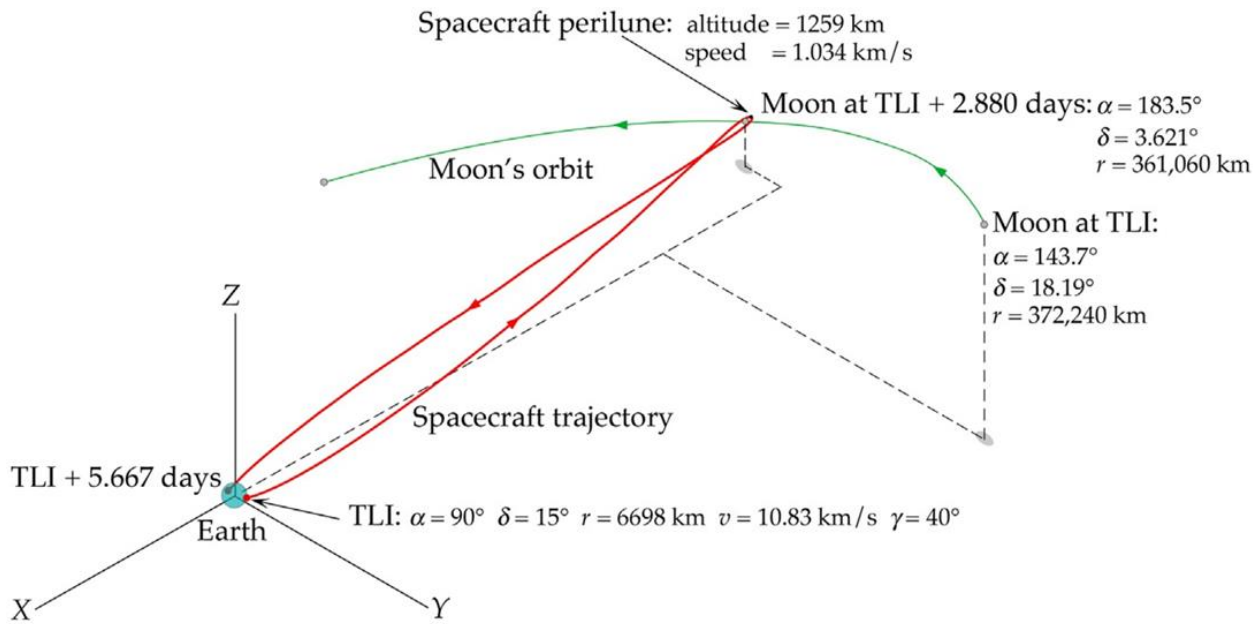
Let $\hat{u}_\perp$ be the unit vector that is normal to both of the orthogonal vectors $\widehat{ur}$ and $\hat{w}$ and therefore lies in the plane of the translunar trajectory. $\hat{u}_\perp$ is simply the cross product of them.

$$\hat{\mathbf{u}}_\perp = \hat{\mathbf{w}} \times \hat{\mathbf{u}}_r$$

We know that $v_{r_0} = v_0 \sin\gamma \quad v_{\perp_0} = v_0 \cos\gamma$

Where γ is initial flight path angle of the spacecraft. Therefore, the velocity vector v0:

$$\mathbf{v}_0 = v_0 \sin\gamma \hat{\mathbf{u}}_r + v_0 \cos\gamma \hat{\mathbf{u}}_\perp$$



4.7. Motion Integration

MATLAB's ode45 Runge - Kutta solver is used.

4.8. Transformation from ECI to moon-fixed frame DCM

We know that:

$$x_{axis} = rm_{mag}$$

$$z_{axis} = rm_{mag}\ x\ vm$$

$$y_{axis} = x_{axis} x z_{axis}$$

Where x_{axis}, y_{axis} , z_{axis} are the vectors of moon frame

$$i_{hat} = \frac{x_{axis}}{x_{axis_{mag}}}$$

$$j_{hat} = \frac{y_{axis}}{y_{axis_{mag}}}$$

$$k_{hat} = \frac{z_{axis}}{z_{axis_{mag}}}$$

Where i_{hat} , j_{hat} , k_{hat} are the unit vectors of the moon fixed frame

Then transformation matrix is formed:

$$\mathrm{Q} = [\mathrm{i_{hat}}, j_{hat}, k_{hat}]$$

Now we can calculate initial position of spacecraft into moon-fixed frame applying this equation:

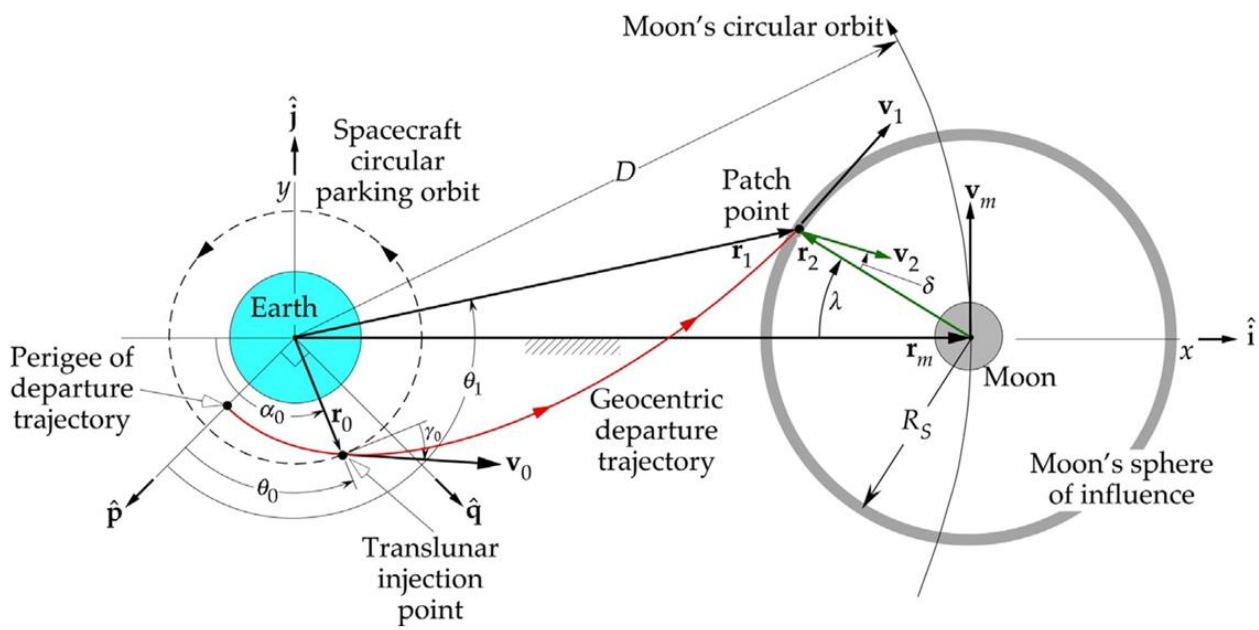
$$\mathrm{r_{inmoonfixed}} = \mathrm{Q} * \mathrm{r}$$

Where r is the spacecraft position in ECI

We can repeat this process with moon's position to convert it in moon-fixed frame.

4.9. Spacecraft's Perilune distance determination

It's the distance of the spacecraft when it's at perigee of the moon's orbit, we're going to find this distance. if you look at this figure, we will find that



$$distance_{vector} = r1 - rm$$

And the current_distance is the magnitude of the $distance_{vector}$

Where r1 here is r in our previous equations. we assumed here that moon's orbit is circular.

By following this method, you can complete the trajectory.

4.10. the history of the trajectory's binormal:

it is the unit vector normal to the osculating plane.

$$\text{binormal} = \frac{\nu \text{ x } a_{\text{total}}}{(\nu \text{ x } a_{\text{total}})_{mag}}$$

where a_{total} is the acceleration of the moon and earth acting on the spacecraft

$$a_{earth} = \frac{-mu_e * r}{r_{mag}^3}$$

$$a_{moon} = mu_m * \left(\frac{r_{ms}}{r_{ms_{mag}}^3} - \frac{r_m}{r_{m_{mag}}^3}\right)$$

5. Trajectory Optimization Function:

In my simulation, I aimed to reduce the **amount of fuel used** during the spacecraft's journey by applying a simple optimization algorithm. The process involved calculating the total **change in velocity (ΔV)** required for the mission and minimizing it by adjusting a scaling factor fac, which influences the velocity required for orbit transfer.

5.1. Before Optimization

I begin by computing the fuel needed for the mission using the nominal (non-optimized) value of the scaling factor fac. This factor affects the calculated ΔV, and therefore the fuel consumption.

```
% compute initial fuel
fuel_before_optimization = compute_fuel_used(delta_v, Isp, g0, m0);
efficiency = (fuel_before_optimization / m0) * 100;
```

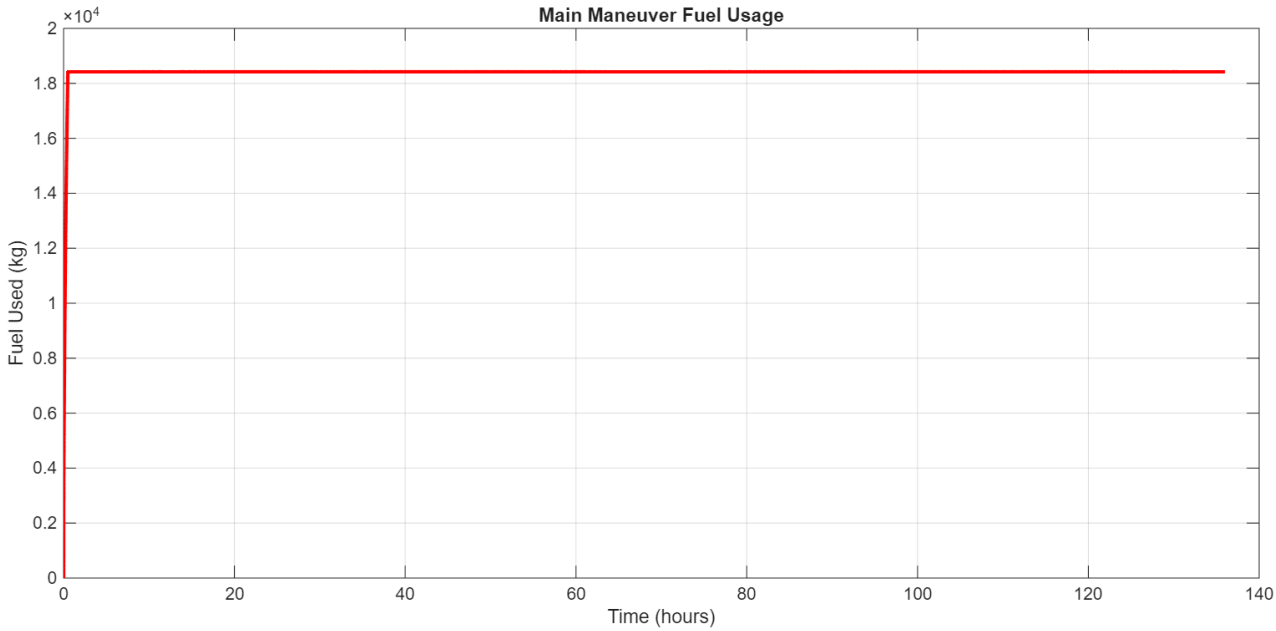
- delta_v: Total required velocity change.
- Isp: Specific impulse of the engine (how efficiently it uses fuel).
- g0: Standard gravity.
- m0: Initial mass of the spacecraft.
- The function compute_fuel_used applies the rocket equation to find the fuel required.
- **Efficiency** is calculated as the ratio of fuel used to initial mass

Output:

```
fprintf('ΔV used (km/s)          = %.4f\n', delta
fprintf('Fuel used (kg)          = %.4f\n', fuel_
fprintf('Efficiency (%%)          = %.2f %%\n', ef
```

Command Window

```
=== Before Optimization ===
fac (nominal)           = 0.99240
ΔV used (km/s)          = 3.1125
Fuel used (kg)          = 18420.9080
Efficiency (%)         = 63.96 %
```



5.2. Applying the Optimization Algorithm

Next, we optimize the value of fac by minimizing the amount of fuel used. This is done using fminbnd, a built-in MATLAB function that finds the minimum of a single-variable function over a specified interval.

```
fac_min = 0.97;
fac_max = 1.01;
objective = @(fac) fuel_used_fac(fac, alt0, mu_e, vesc, Re, Isp, m0, g0);
[fac_opt, min_fuel_used] = fminbnd(objective, fac_min, fac_max);
```

- The objective function calculates the fuel used based on the value of fac.
- The optimizer searches for the best fac value between 0.97 and 1.01 that minimizes fuel consumption.
- We then recompute the optimized ΔV and efficiency.

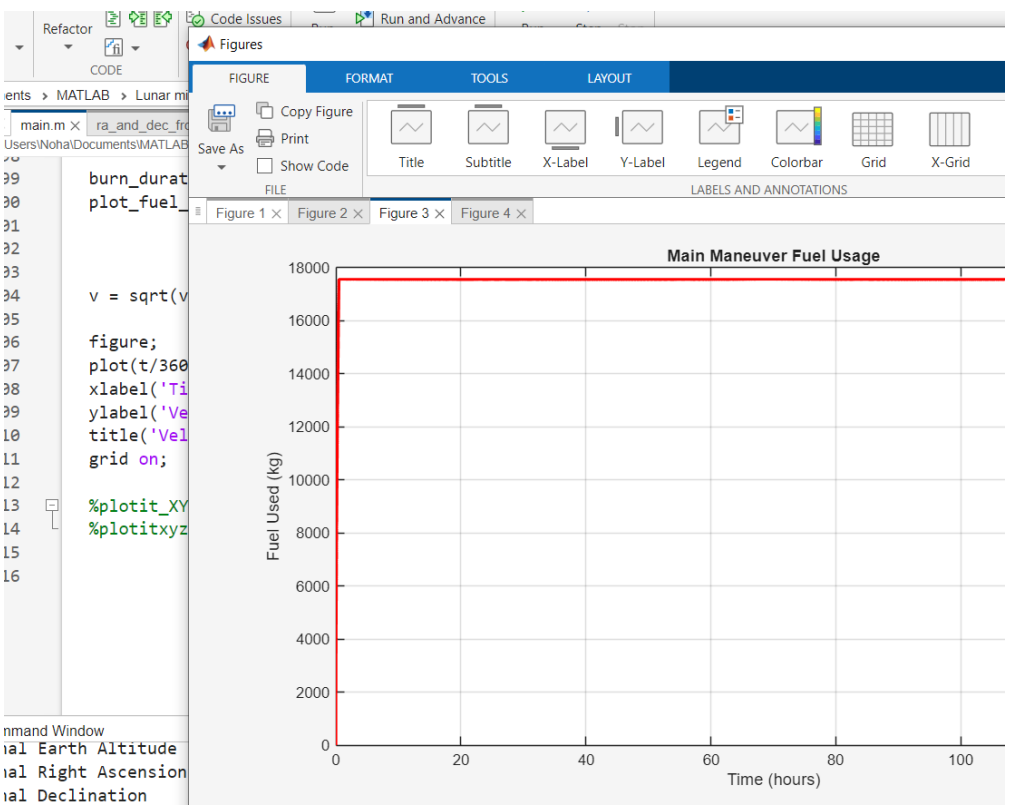
```
[deltaV_opt, ~] = compute_total_deltavandfuel(alt0, mu_e, fac_opt, vesc, Re,
Isp, m0, g0);
opt_efficiency = (min_fuel_used / m0) * 100;
```

5.3. After Optimization

```
figure;
plot(t/3600, v, 'b', 'LineWidth', 2
xlabel('Time (hours)');
```

Command Window

```
Start


=== Before Optimization ===
fac (nominal)              = 0.99240
ΔV used (km/s)             = 3.1125
Fuel used (kg)             = 18420.9080
Efficiency (%)            = 63.96 %

=== Optimization Results ===
Optimal fac                 = 0.97004
ΔV (km/s)                   = 2.8685
Fuel used (kg)              = 17556.7006
Efficiency (%)             = 60.96 %
```



5.4. Impact of Optimization:

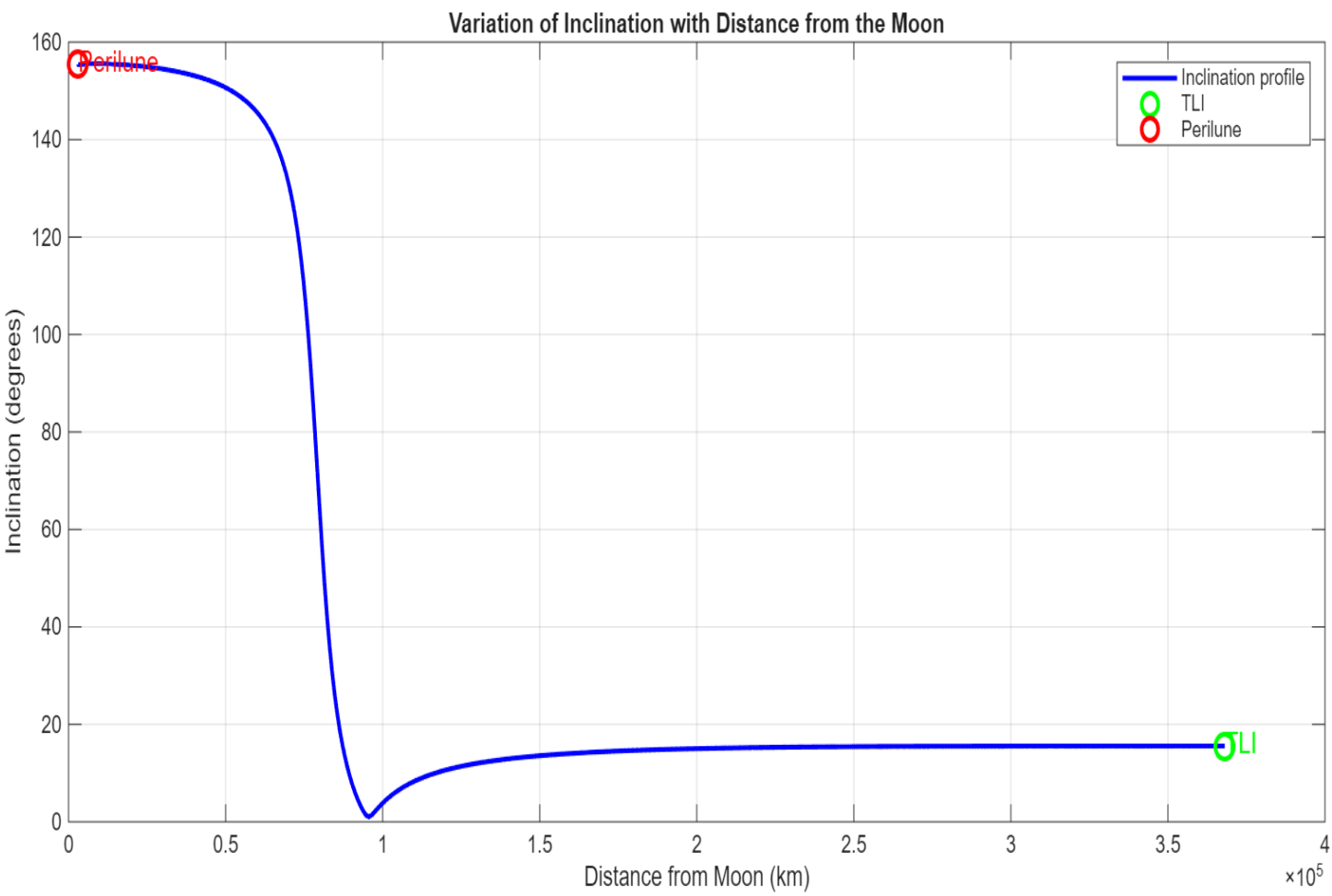
- The ΔV was slightly reduced.
- Fuel consumption dropped by ~860 kg.
- This small difference leads to significant savings in space missions, where every kilogram of fuel is expensive.

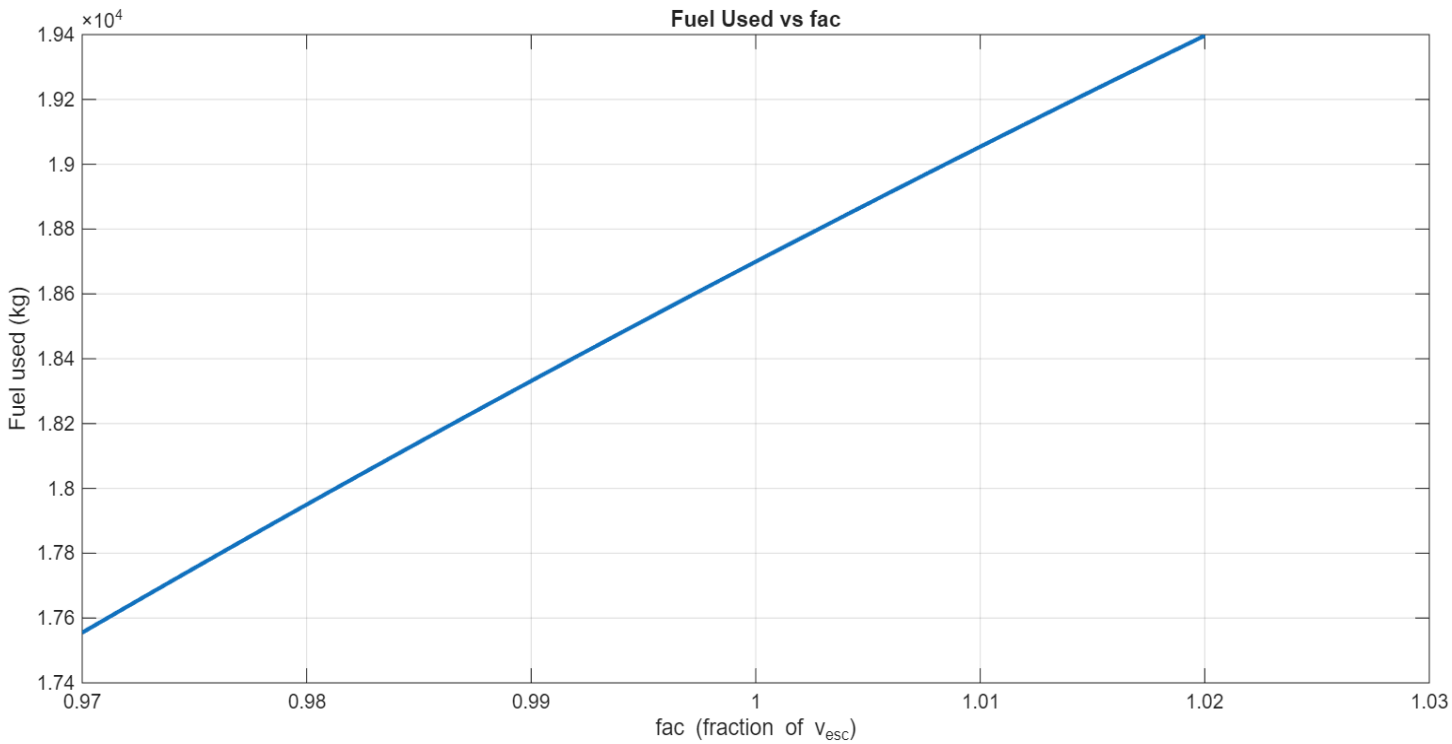
By integrating such optimization into mission planning, we ensure a **more efficient and cost-effective spaceflight**

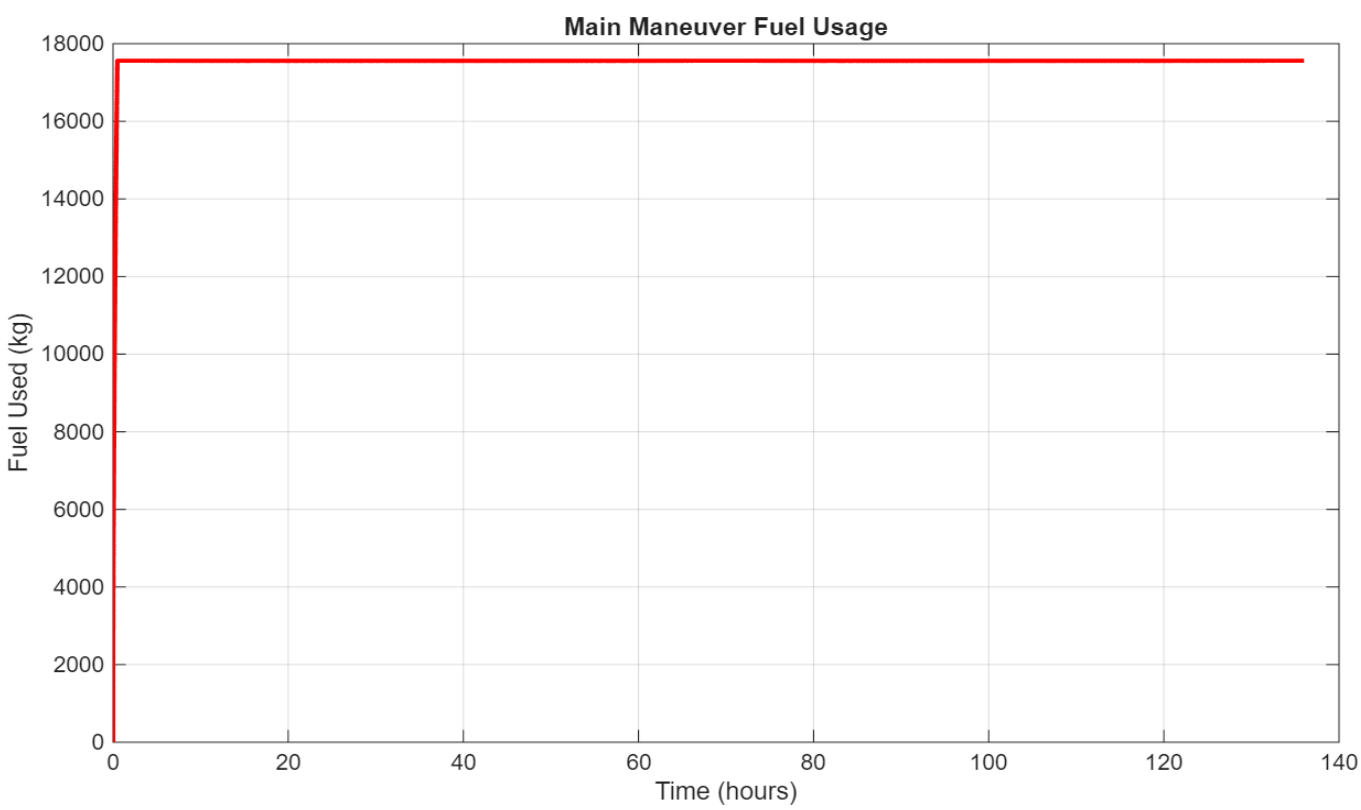
6. Results and Outputs

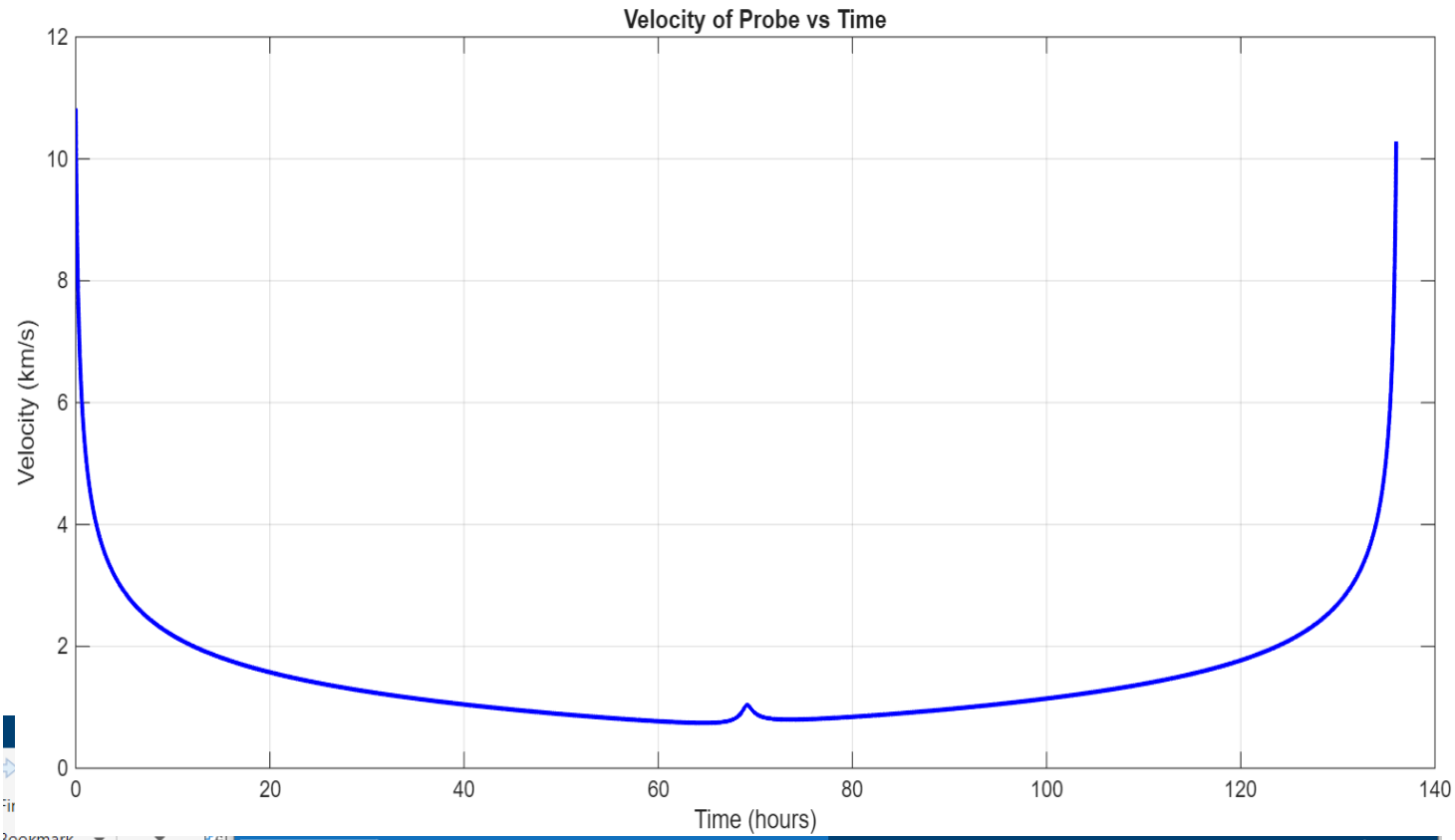
```
=== Before Optimization ===
fac (nominal)           = 0.99240
ΔV used (km/s)          = 3.1125
Fuel used (kg)          = 18420.9080
Efficiency (%)           = 63.96 %

=== Optimization Results ===
Optimal fac              = 0.97004
ΔV (km/s)                = 2.8685
Fuel used (kg)           = 17556.7006
Efficiency (%)           = 60.96 %
 distance             =      360785 km
 RA                   =     185.107 deg
 Dec                  =     2.91682 deg
 inclmoon            =     23.6765 deg

The probe at Earth departure (t = 0 sec):
 Altitude             =         320 km
 Right Ascension      =          90 deg
 Declination          =          15 deg
 Flight Path Angle    =          40 deg
 Speed                =     10.8267 km/s
 Escape Speed         =     10.9097 km/s
 v / v_esc            =      0.9924
 Inclination of Translunar Orbit =     15.5505 deg
 Distance             =      372242 km
 Right Ascension      =     143.743 deg
 Declination          =      18.189 deg

The moon when the probe is at perilune:
 Distance             =      361064 km
 Speed                =     1.07674 km/s
 Right Ascension      =     183.483 deg
 Declination          =     3.62073 deg
 Target Error         =     11136.8 km

The probe at Perilune:
 Altitude above Moon        =   1258.9427 km
 Absolute Speed             =      1.0346 km/s
```

```
 Relative Speed to Moon      =      2.1105 km/s
 Inclination of Osculating Plane =    155.3929 deg

Time from TLI to Perilune =     69.1276 hours (2.8803 days)
Total Time of Flight          =      5.6670 days
Time to Target Point          =      3.0000 days

Final Earth Altitude        =   1035.4337 km
Final Right Ascension         =    263.6075 deg
Final Declination             =    -27.4236 deg
```




Fuel Used vs fac
×10^4
1.94
1.92
1.9
1.88
1.86
1.84
1.82
1.8
1.78
1.76
1.74
Fuel used (kg)
0.97
0.98
0.99
1
1.01
1.02
1.03
fac (fraction of v_esc)


Main Maneuver Fuel Usage
18000
16000
14000
12000
10000
8000
6000
4000
2000
0
Fuel Used (kg)
0
20
40
60
80
100
120
140
Time (hours)


Velocity of Probe vs Time
Velocity (km/s)
Time (hours)
0
2
4
6
8
10
12
0
20
40
60
80
100
120
140

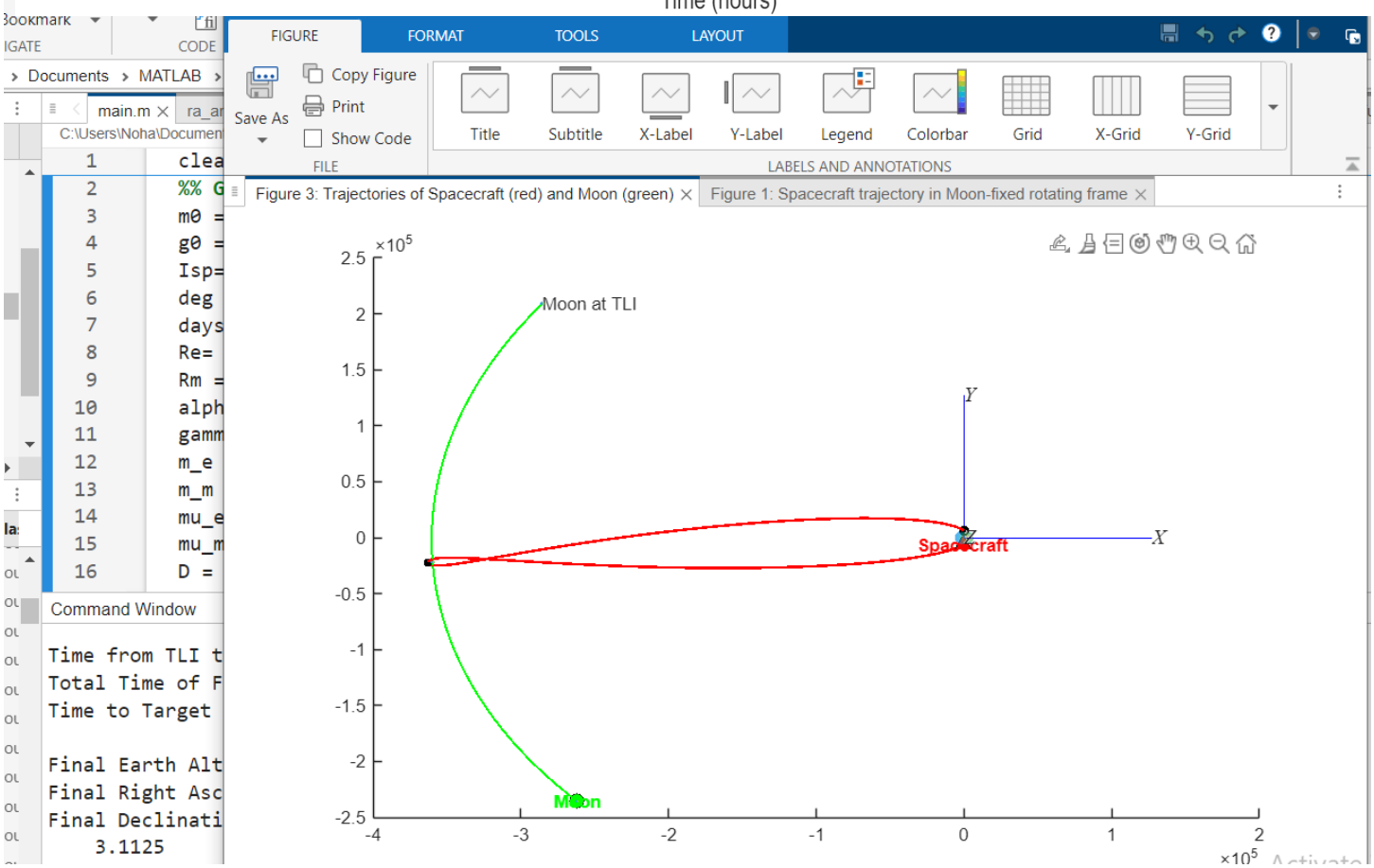
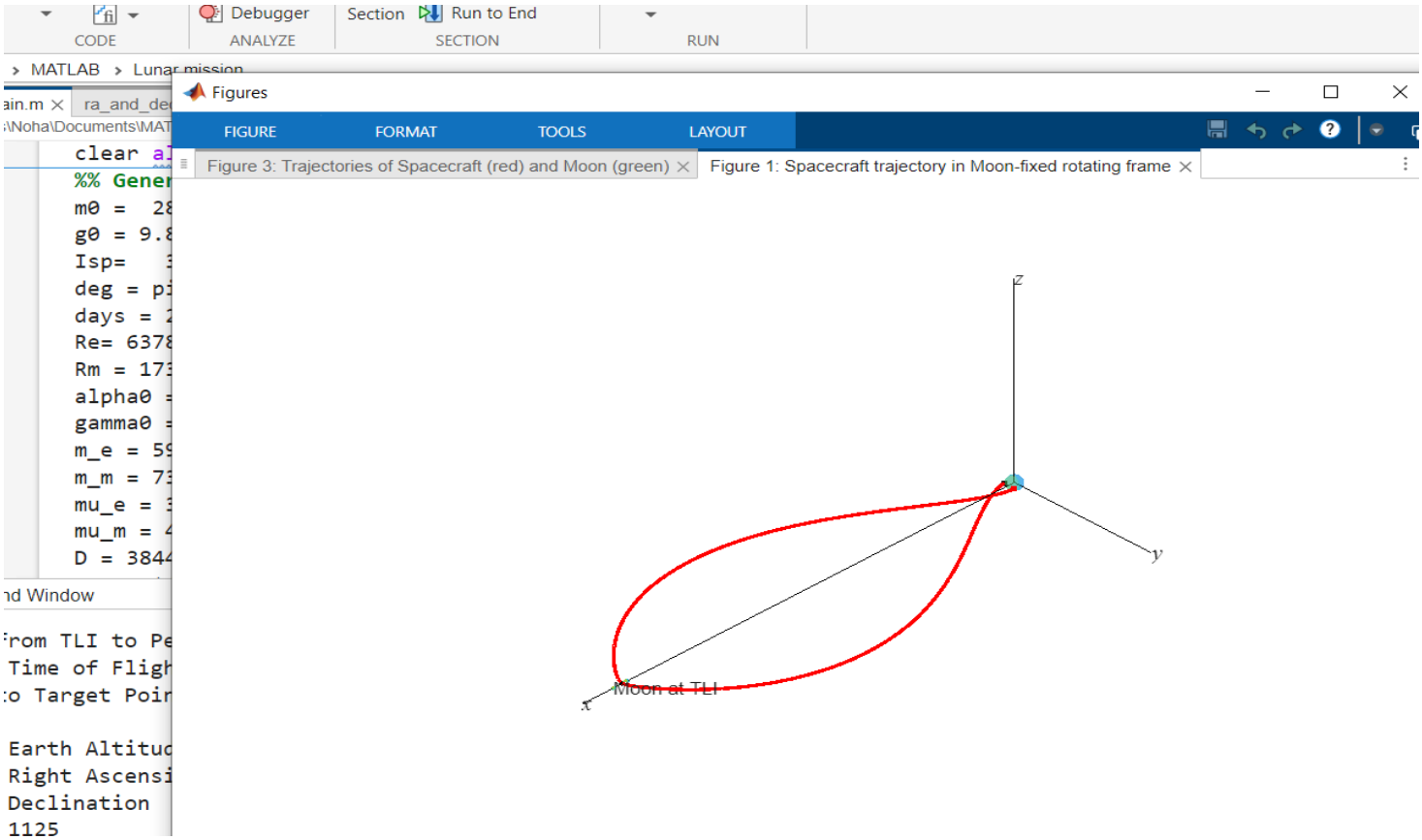


FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 3: Trajectories of Spacecraft (red) and Moon (green)
Figure 1: Spacecraft trajectory in Moon-fixed rotating frame
Moon at TLI
Spacecraft
Moon
Y
X
×10^5
Command Window
Time from TLI t
Total Time of F
Time to Target
Final Earth Alt
Final Right Asc
Final Declinati
3.1125



7. MATLAB Script

```
clear all; close all; clc
%% General data
m0 =  28801;
g0 = 9.80665;
Isp=   311;
deg = pi/180;
days = 24*3600;
Re= 6378;
Rm = 1737;
alpha0 =90;
gamma0 = 40;
m_e = 5974.e21;
m_m = 73.48e21;
mu_e = 398600.4;
mu_m = 4902.8;
D = 384400;
RS = D*(m_m/m_e)^(2/5);
Title = 'Start';
fac = .9924;
```

```
ttt = 3*days;
tf = ttt+ 2.667*days;
dec0 = 15;
alt0 =320;
z0 = alt0+Re;
speed_at_LEO = sqrt(mu_e / z0);
vesc = sqrt(2* mu_e / z0);
speed_TLI= fac * vesc;
delta_v = speed_TLI - speed_at_LEO;

% compute initial fuel
fuel_before_optimization = compute_fuel_used(delta_v, Isp, g0, m0);
efficiency = (fuel_before_optimization / m0) * 100;

% Output
fprintf('\n\n%s\n\n', Title)

fprintf('\n=== Before Optimization ===\n');
fprintf('fac (nominal)           = %.5f\n', fac);
fprintf('ΔV used (km/s)          = %.4f\n', delta_v);
fprintf('Fuel used (kg)          = %.4f\n', fuel_before_optimization);
fprintf('Efficiency (%%)         = %.2f %%\n', efficiency);

% === Optimization ===
fac_min = 0.97;
fac_max = 1.01;
objective = @(fac) fuel_used_fac(fac, alt0, mu_e, vesc, Re, Isp, m0, g0);
[fac_opt, min_fuel_used] = fminbnd(objective, fac_min, fac_max);
[deltaV_opt, ~] = compute_total_deltavandfuel(alt0, mu_e, fac_opt, vesc,
Re, Isp, m0, g0);
opt_efficiency = (min_fuel_used / m0) * 100;

fprintf('\n=== Optimization Results ===\n');
fprintf('Optimal fac              = %.5f\n', fac_opt);
fprintf('ΔV (km/s)                = %.4f\n', deltaV_opt);
fprintf('Fuel used (kg)           = %.4f\n', min_fuel_used);
fprintf('Efficiency (%%)          = %.2f %%\n', opt_efficiency);

% === Plot fuel vs fac ===
fac_vals = linspace(0.97, 1.02, 200);
fuel_vals = arrayfun(@(f) compute_fuel_used(f*vesc - speed_at_LEO, Isp,
g0, m0), fac_vals);

%figure;
plot(fac_vals, fuel_vals, 'LineWidth', 2);
xlabel('fac (fraction of v_{esc})');
```

```
ylabel('Fuel used (kg)');
title('Fuel Used vs fac');
grid on;

%% Date and time of lunar arrival:
year = 2020;
month = 5;
day= 4;
hour = 12;
minute = 0;
second = 0;
UT =12;
t0 =0;
jd0 = julian_day(year, month, day, UT);

%% state vector of the moon at target date

[rm0,vm0] = simpsons_lunar_ephemeris(jd0);
[RA, Dec] =  ra_and_dec_from_r(rm0);
distance =norm(rm0);
hmoon = cross(rm0,vm0);
hmoon_mag = norm(hmoon);
inclmoon = acosd(hmoon(3)/hmoon_mag);

fprintf('Date and time of arrival at moon: ')
fprintf('%s/%s/%s %s:%s:%s', ...
num2str(month), num2str(day), num2str(year), ...
num2str(hour), num2str(minute), num2str(second))
fprintf('\nMoon''s position: ')
fprintf(' distance             = %11g km\n', distance);
fprintf(' RA                   = %11g deg\n', RA);
fprintf(' Dec                  = %11g deg\n', Dec);
fprintf(' inclmoon             = %11g deg\n', inclmoon);

%% ...Initial position vector of probe:
I = [1;0;0];
J = [0;1;0];
K = cross(I,J);
r0_mag = Re+alt0;
r0 = r0_mag*(cosd(alpha0)*cosd(dec0)*I + ...
 sind(alpha0)*cosd(dec0)*J + ...
 sind(dec0)*K);

w0 =  cross(r0,rm0)/norm(cross(r0,rm0));

fprintf('\nThe probe at Earth departure (t = %g sec):\n', t0);
```

```
fprintf(' Altitude              = %11g km\n', alt0);
fprintf(' Right Ascension       = %11g deg\n', alpha0);
fprintf(' Declination           = %11g deg\n', dec0);
fprintf(' Flight Path Angle     = %11g deg\n', gamma0);
fprintf(' Speed                 = %11g km/s\n', speed_TLI);
fprintf(' Escape Speed          = %11g km/s\n', vesc);
fprintf(' v / v_esc             = %11g\n', speed_TLI/vesc);
fprintf(' Inclination of Translunar Orbit = %11g deg\n', acosd(w0(3)));

 %% ...Initial velocity vector of probe:
 ur = r0/norm(r0);
 uperp = cross(w0,ur)/norm(cross(w0,ur));
 vr_mag =speed_TLI*sind(gamma0);
 vperp_mag = speed_TLI*cosd(gamma0);
 v0 = vr_mag*ur + vperp_mag*uperp;
 uv0 = v0 /speed_TLI;
%...Initial state vector of the probe:
 y0 = [r0(1) r0(2) r0(3) v0(1) v0(2) v0(3)]';

%% ODE integration
options = odeset('RelTol', 1.e-10, 'AbsTol', 1.e-10,'Stats', 'off');
rates_func = @(t, y) rates(t, y, jd0, ttt, days, mu_m, mu_e);
[t, y] = ode45(rates_func, [t0 tf], y0, options);
%[r0 ,v0, y0,w0_hat] = launch_conditions2rv(Re,dec0, alt0, RAAN0,
gamma0,v0_mag,rm0);

%% ...Spacecraft trajectory
 % in ECI frame:
 X =y(:,1); Y = y(:,2); Z = y(:,3);
 vX = y(:,4); vY = y(:,5); vZ = y(:,6);
 % in Moon-fixed frame:
 x =[]; y =[]; z =[];
 %% ...Moon trajectory
 % in ECI frame:
 Xm = []; Ym = []; Zm = [];
 vXm = []; vYm = []; vZm = [];
 % in Moon-fixed frame:
 xm = []; ym = []; zm = [];

%Starting value in the search for perilune
dist_min = 1e30;

for i = 1:length(t)
    ti = t(i);

    %...Probe's inertial position vector at time ti
```

```
    r = [X(i); Y(i); Z(i)];

     %...Moon's inertial position and velocity vectors at time ti:
    jd = jd0 - (ttt - ti) / days;
    [rm_mag, vm] = simpsons_lunar_ephemeris(jd);

    %...Moon's inertial state vector at time ti:
    Xm = [Xm; rm_mag(1)];
    Ym = [Ym; rm_mag(2)];
    Zm = [Zm; rm_mag(3)];
    vXm = [vXm; vm(1)];
    vYm = [vYm; vm(2)];
    vZm = [vZm; vm(3)];

     %...Moon-fixed rotating xyz frame:
    x_axis = rm_mag;
    z_axis = cross(rm_mag, vm);
    y_axis = cross(z_axis, x_axis);
    i_hat = x_axis / norm(x_axis);
    j_hat = y_axis / norm(y_axis);
    k_hat = z_axis / norm(z_axis);

    %...DCM of transformation from ECI to moon-fixed frame:
    Q = [i_hat'; j_hat'; k_hat'];

    %...Components of probe's inertial position vector in moon-fixed
frame:
    r_moon_fixed = Q * r;
    x = [x; r_moon_fixed(1)];
    y = [y; r_moon_fixed(2)];
    z = [z; r_moon_fixed(3)];

    %...Components of moon's inertial position vector in moon-fixed frame:
    rm_moon_fixed = Q * rm_mag;
    xm = [xm; rm_moon_fixed(1)];
    ym = [ym; rm_moon_fixed(2)];
    zm = [zm; rm_moon_fixed(3)];

     %...Find perilune of the probe:
    distance_vector = r - rm_mag;
    current_distance = norm(distance_vector);

    if current_distance < dist_min
        imin = i;
        dist_min = current_distance;
    end
```

```
end

 %...Location of the Moon at TLI:
rmTLI = [Xm(1); Ym(1); Zm(1)];
[RATLI, DecTLI] = ra_and_dec_from_r(rmTLI);
fprintf('\nThe moon when the probe is at TLI:')
fprintf(' Distance             = %11g km\n', norm(rmTLI));
fprintf(' Right Ascension      = %11g deg\n', RATLI);
fprintf(' Declination          = %11g deg\n', DecTLI);

%...State vector and celestial position of moon when probe is at perilune:
rm_perilune = [Xm(imin); Ym(imin); Zm(imin)];
vm_perilune = [vXm(imin); vYm(imin); vZm(imin)];
[RA_perilune, Dec_perilune] = ra_and_dec_from_r(rm_perilune);

 %...Spacecraft velocity at perilune:
v_atdmin =[vX(imin); vY(imin); vZ(imin) ];
 %...State vector and celestial position of moon when probe is at
perilune:
rm_perilune = [Xm(imin); Ym(imin); Zm(imin)];
vm_perilune = [vXm(imin); vYm(imin); vZm(imin)];
[RA_at_perilune,Dec_at_perilune] = ra_and_dec_from_r(rm_perilune);
target_error = norm(rm_perilune - rm0);

fprintf('\nThe moon when the probe is at perilune:\n');
fprintf(' Distance             = %11g km\n', norm(rm_perilune));
fprintf(' Speed                = %11g km/s\n', norm(vm_perilune));
fprintf(' Right Ascension      = %11g deg\n', RA_at_perilune);
fprintf(' Declination          = %11g deg\n', Dec_at_perilune);
fprintf(' Target Error         = %11g km\n', target_error);


 %...Speed of probe relative to Moon at perilune:
rel_speed = norm(v_atdmin - vm_perilune);

 %...End point of trajectory:
rend = [X(end); Y(end); Z(end)];
alt_end = norm(rend) - Re;
[ra_end, dec_end] = ra_and_dec_from_r(rend);

 %...Find the history of the trajectory's binormal:
for i = 1:imin
    time(i)     = t(i);

    r_vec       = [X(i); Y(i); Z(i)];
    r_mag       = norm(r_vec);
```

```
    v_vec       = [vX(i); vY(i); vZ(i)];

    rm_vec      = [Xm(i); Ym(i); Zm(i)];
    rm_mag      = norm(rm_vec);

    rms_vec     = rm_vec - r_vec;
    rms_mag     = norm(rms_vec);

    a_earth     = -mu_e * r_vec / r_mag^3;
    a_moon      = mu_m * (rms_vec / rms_mag^3 - rm_vec / rm_mag^3);

    atotal      = a_earth + a_moon;

    binormal    = cross(v_vec, atotal) / norm(cross(v_vec, atotal));
    binormal_z  = binormal(3);

    incl(i)     = acosd(binormal_z);
    moon_distance(i) = norm(r_vec - rm_vec);  % distance from the moon

end

figure;
plot(moon_distance, incl, 'b', 'LineWidth', 2);
hold on;

plot(moon_distance(1), incl(1), 'go', 'MarkerSize', 10, 'LineWidth', 2); %
TLI
text(moon_distance(1), incl(1)+0.5, 'TLI', 'Color', 'green', 'FontSize',
12);

plot(moon_distance(imin), incl(imin), 'ro', 'MarkerSize', 10, 'LineWidth',
2); % Perilune
text(moon_distance(imin), incl(imin)+0.5, 'Perilune', 'Color', 'red',
'FontSize', 12);

xlabel('Distance from Moon (km)');
ylabel('Inclination (degrees)');
title('Variation of Inclination with Distance from the Moon');
grid on;
legend('Inclination profile','TLI','Perilune');


%%... Output
fprintf('\nThe probe at Perilune:\n');
fprintf(' Altitude above Moon        = %11.4f km\n', dist_min - Rm);
```

```
fprintf(' Absolute Speed              = %11.4f km/s\n', norm(v_atdmin));
fprintf(' Relative Speed to Moon      = %11.4f km/s\n', rel_speed);
fprintf(' Inclination of Osculating Plane = %11.4f deg\n', incl(imin));

fprintf('\nTime from TLI to Perilune = %11.4f hours (%.4f days)\n',
abs(t(imin))/3600, abs(t(imin))/(3600*24));
fprintf('Total Time of Flight          = %11.4f days\n', t(end)/days);
fprintf('Time to Target Point          = %11.4f days\n', ttt/days);

fprintf('\nFinal Earth Altitude        = %11.4f km\n', alt_end);
fprintf('Final Right Ascension         = %11.4f deg\n', ra_end);
fprintf('Final Declination             = %11.4f deg\n', dec_end);
 %...End output

deltaV_total = delta_v;
burn_duration = 1800;
plot_fuel_vs_time(t, m0, deltaV_total, burn_duration, Isp);


v = sqrt(vX.^2 + vY.^2 + vZ.^2);

figure;
plot(t/3600, v, 'b', 'LineWidth', 2);
xlabel('Time (hours)');
ylabel('Velocity (km/s)');
title('Velocity of Probe vs Time');
grid on;

plotit_XYZ(X, Y, Z, Xm, Ym, Zm, imin, Re, Rm)
plotitxyz(x, y, z, xm, ym, zm, imin,Re,Rm)

function [ra, dec] = ra_and_dec_from_r(r)
    l = r(1)/norm(r);
    m = r(2)/norm(r);
    n = r(3)/norm(r);
    dec = asind(n);

    if m > 0
         ra = acosd(l/cosd(dec));
    else
         ra = 360- acosd(l/cosd(dec));
    end

function [pos,vel] = simpsons_lunar_ephemeris(jd)

% Constants
```

```
tfac = 36525*3600*24;                    % seconds in a Julian century
t = (jd - 2451545.0) / 36525;             % centuries since J2000
% Amplitude matrix a (in km)
a = [383.0  31.5  10.6  6.2  3.2  2.3  0.8;
     351.0  28.9  13.7  9.7  5.7  2.9  2.1;
     153.2  31.5  12.5  4.2  2.5  3.0  1.8] * 1e3;
% Frequency matrix b (in rad/century)
b = [ 8399.685  70.990  16728.377  1185.622  7143.070  15613.745
8467.263;
      8399.687  70.997  8433.466  16728.380  1185.667  7143.058
15613.755;
      8399.672  8433.464 70.996  16728.364  1185.645  104.881  8399.116];
% Phase angle matrix c (in radians)
c = [5.381 6.169 1.453 0.481 5.017 0.857 1.010;
     3.811 4.596 4.766 6.165 5.164 0.300 5.565;
     3.807 1.629 4.595 6.162 5.167 2.555 6.248];
% Initialize vectors
pos = zeros(3, 1);
vel = zeros(3, 1);
% Compute position and velocity vectors
for i = 1:3
    for j = 1:7
        angle = b(i, j) * t + c(i, j);
        pos(i) = pos(i) + a(i, j) * sin(angle);
        vel(i) = vel(i) + a(i, j) * cos(angle) * b(i, j);
    end
    vel(i) = vel(i) / tfac;
end
% Display results
%fprintf('Moon Position Vector (km):\n');
%fprintf('Moon position vector at arrival (rm0) = [%.2f, %.2f, %.2f]\n',
rm0(1), rm0(2), rm0(3));
%fprintf('moon distance at arrival: %.3f km\n', rm_mag);
%fprintf('Moon velocity vector at arrival (vm0) = [%.4f, %.4f, %.4f]\n',
vm0(1), vm0(2), vm0(3));
%ym0 = [rm0(1), rm0(2), rm0(3), vm0(1), vm0(2), vm0(3)];
%fprintf('moon state vector at arrival in ECI frame (y0m) = [%.4f, %.4f,
%.4f, %.4f, %.4f, %.4f]\n', ym0);
end

function [r0, v0, y0,w0_hat] = launch_conditions2rv(Re, dec0, alt0, RAAN0,
gamma0,v0_mag,rm0)
    % Calculate the r0 vector at Trans-Lunar Injection (TLI)
    % r0 is the initial position vector of lunar trajectory insertion
    % Convert angles from degrees to radians
    dec_rad = deg2rad(dec0);
```

```
    RAAN_rad = deg2rad(RAAN0);
    gamma_rad = deg2rad(gamma0);
    % Compute position vector
    r0_mag = alt0 + Re;
    r0_x = r0_mag * cos(dec_rad) * cos(RAAN_rad);
    r0_y = r0_mag * cos(dec_rad) * sin(RAAN_rad);
    r0_z = r0_mag * sin(dec_rad);
    r0 = [r0_x, r0_y, r0_z];  % row vector
    ur_hat = r0/norm(r0);
    w0_hat = cross(r0,rm0)/ norm(cross(r0,rm0));
    ut_hat = cross (w0_hat,ur_hat);
    v0 = v0_mag * sin(gamma_rad) * ur_hat + v0_mag * cos(gamma_rad) *
ut_hat;

    % Print vector as [x, y, z] format
    %fprintf('initial position vector of Probe (r0) = [%.2f, %.2f, %.2f]
km\n', r0);
    %fprintf('initial velocity vector of Probe (v0) = [%.4f, %.4f, %.4f]
km/s\n', v0);
    y0 = [r0_x, r0_y, r0_z v0 ];
    %fprintf('initial state vector of Probe in ECI frame (y0_prb) = [%.4f,
%.4f, %.4f, %.4f, %.4f, %.4f]\n', y0);

end

function [jd] = julian_day(year, month, day,UT)

j0 = 367*year- fix(7*(year + fix((month + 9)/12))/4) ...
 + fix(275*month/9) + day + 1721013.5;
jd =   j0 + UT/24;

end

function dydt = rates(t, y, jd0, ttt, days, mu_m, mu_e)
jd = jd0 -(ttt-t)/days;
X = y(1);
Y = y(2);
Z = y(3);
vX = y(4);
vY = y(5);
vZ = y(6);
r = [X Y Z ]';
r_mag =norm(r);
[rm,~] = simpsons_lunar_ephemeris(jd);
rm_mag = norm(rm);
rms = rm-r;
```

```
rms_mag = norm(rms);
a_earth = -mu_e *r /r_mag^3;
a_moon = mu_m*(rms/rms_mag^3 - rm/rm_mag^3);
a = a_earth + a_moon;
aX= a(1);
aY= a(2);
aZ= a(3);
dydt = [vX vY vZ aX aY aZ]';
%y = [X; Y; Z vX; vY; vZ];
%fprintf('y = [%.4f, %.4f, %.4f, %.4f, %.4f, %.4f]\n', y);
%fprintf('dydt = [%.4f, %.4f, %.4f, %.4f, %.4f, %.4f]\n', dydt);
end

function plotit_XYZ(X, Y, Z, Xm, Ym, Zm, imin, Re, Rm)
%-------------------------------------

figure('Name','Trajectories of Spacecraft (red) and Moon (green)',
'Color', 'w');


[xx, yy, zz] = sphere(128);

hold on

%... Geocentric inertial coordinate axes
L = 20 * Re;
line([0 L], [0 0], [0 0], 'color','b')
text(L, 0, 0, 'X', 'FontSize', 12, 'FontAngle', 'italic', 'FontName',
'Palatino')
line([0 0], [0 L], [0 0], 'color','b')
text(0, L, 0, 'Y', 'FontSize', 12, 'FontAngle', 'italic', 'FontName',
'Palatino')
line([0 0], [0 0], [0 L], 'color','b')
text(0, 0, L, 'Z', 'FontSize', 12, 'FontAngle', 'italic', 'FontName',
'Palatino')

%... Earth
Earth = surfl(Re*xx, Re*yy, Re*zz);
set(Earth, 'FaceAlpha', 0.5);
shading interp

%... Spacecraft at TLI
plot3(X(1), Y(1), Z(1), 'o', 'MarkerEdgeColor', 'k', 'MarkerFaceColor',
'k', 'MarkerSize', 5)

%... Spacecraft at closest approach (perilune)
```

```
plot3(X(imin), Y(imin), Z(imin), 'o', 'MarkerEdgeColor', 'k',
'MarkerFaceColor', 'k', 'MarkerSize', 4)

%... Spacecraft at final time
plot3(X(end), Y(end), Z(end), 'o', 'MarkerEdgeColor', 'r',
'MarkerFaceColor', 'r', 'MarkerSize', 5)

%... Moon at TLI
text(Xm(1), Ym(1), Zm(1), 'Moon at TLI')
Moon1 = surfl(Rm*xx + Xm(1), Rm*yy + Ym(1), Rm*zz + Zm(1));
set(Moon1, 'FaceAlpha', 0.99)
shading interp

%... Moon at closest approach
Moon2 = surfl(Rm*xx + Xm(imin), Rm*yy + Ym(imin), Rm*zz + Zm(imin));
set(Moon2, 'FaceAlpha', 0.99)
shading interp

%... Moon at end of simulation
Moon3 = surfl(Rm*xx + Xm(end), Rm*yy + Ym(end), Rm*zz + Zm(end));
set(Moon3, 'FaceAlpha', 0.99)
shading interp

%... Spacecraft trajectory
%plot3(X, Y, Z, 'r', 'LineWidth', 1.5)

%... Moon trajectory
%plot3(Xm, Ym, Zm, 'g', 'LineWidth', 1)
% Create animated lines
spacecraftLine = animatedline('Color','r','LineWidth',1.5);
moonLine       = animatedline('Color','g','LineWidth',1.2);

% Animation loop
for k = 1:length(X)
     addpoints(spacecraftLine, X(k), Y(k), Z(k));
    addpoints(moonLine, Xm(k), Ym(k), Zm(k));


    h1 = plot3(X(k), Y(k), Z(k), '^', 'MarkerFaceColor', 'r', ...
               'MarkerEdgeColor', 'k', 'MarkerSize', 8);


    t1 = text(X(k), Y(k), Z(k) + 0.1*Re, 'Spacecraft', ...
              'FontSize', 10, 'FontWeight','bold', ...
              'Color', 'r', 'HorizontalAlignment','center');
```

```
    h2 = plot3(Xm(k), Ym(k), Zm(k), 'o', 'MarkerFaceColor', 'g', ...
               'MarkerEdgeColor', 'k', 'MarkerSize', 8);


    t2 = text(Xm(k), Ym(k), Zm(k) + 0.1*Re, 'Moon', ...
              'FontSize', 10, 'FontWeight','bold', ...
              'Color', 'g', 'HorizontalAlignment','center');

    drawnow limitrate

    % Remove current markers and labels
    if k < length(X)
        delete(h1); delete(h2);
        delete(t1); delete(t2);
    end
end

function plotitxyz(x, y, z, xm, ym, zm, imin,Re,Rm)

figure('Name', 'Spacecraft trajectory in Moon-fixed rotating frame', ...
       'Color', [1 1 1]);

[xx, yy, zz] = sphere(128);
hold on

%... Spacecraft trajectory:
plot3(x, y, z, 'r', 'LineWidth', 2.0)

%... Moon trajectory:
plot3(xm, ym, zm, 'g', 'LineWidth', 0.5)

%... Earth:
Earth = surfl(Re * xx, Re * yy, Re * zz);
set(Earth, 'FaceAlpha', 0.5);
shading interp

%... Geocentric moon-fixed coordinate axes:
L1 = 63 * Re;
L2 = 20 * Re;
L3 = 29 * Re;

line([0 L1], [0 0], [0 0], 'color', 'k')
text(L1, 0, 0, 'x', 'FontSize', 12, 'FontAngle', 'italic', 'FontName',
'Palatino')
```

```
line([0 0], [0 L2], [0 0], 'color', 'k')
text(0, L2, 0, 'y', 'FontSize', 12, 'FontAngle', 'italic', 'FontName',
'Palatino')

line([0 0], [0 0], [0 L3], 'color', 'k')
text(0, 0, L3, 'z', 'FontSize', 12, 'FontAngle', 'italic', 'FontName',
'Palatino')

%... Spacecraft at TLI
plot3(x(1), y(1), z(1), 'o', ...
      'MarkerEdgeColor', 'k', 'MarkerFaceColor', 'k', 'MarkerSize', 3)

%... Spacecraft at closest approach
plot3(x(imin), y(imin), z(imin), 'o', ...
      'MarkerEdgeColor', 'k', 'MarkerFaceColor', 'k', 'MarkerSize', 2)

%... Spacecraft at tf (end)
plot3(x(end), y(end), z(end), 'o', ...
      'MarkerEdgeColor', 'r', 'MarkerFaceColor', 'r', 'MarkerSize', 3)

%... Moon at TLI
text(xm(1), ym(1), zm(1), 'Moon at TLI')
Moon1 = surfl(Rm * xx + xm(1), Rm * yy + ym(1), Rm * zz + zm(1));
set(Moon1, 'FaceAlpha', 0.99)
shading interp

%... Moon at closest approach
Moon2 = surfl(Rm * xx + xm(imin), Rm * yy + ym(imin), Rm * zz + zm(imin));
set(Moon2, 'FaceAlpha', 0.99)
shading interp

%... Moon at end of simulation
Moon3 = surfl(Rm * xx + xm(end), Rm * yy + ym(end), Rm * zz + zm(end));
set(Moon3, 'FaceAlpha', 0.99)
shading interp

axis image
axis vis3d
axis off
view([1, 1, 1])
end

function fuel = compute_fuel_used(deltaV, Isp, g0, m0)
    if deltaV <= 0
        fuel = 0;
        return;
```

```
    end
    deltaV_mps = deltaV * 1000;
    mf = m0 / exp(deltaV_mps / (Isp * g0));
    fuel = m0 - mf;
end

function fuel = fuel_used_fac(fac, alt0, mu_e, vesc, Re, Isp, m0, g0)
    r0 = Re + alt0;
    v_circular = sqrt(mu_e / r0);
    v_TLI = fac * vesc;
    deltaV = v_TLI - v_circular;

    fuel = compute_fuel_used(deltaV, Isp, g0, m0);
end

function plot_fuel_vs_time(t, m0, deltaV_total, burn_duration, Isp)
    g0 = 9.80665;  % m/s^2
    deltaV_mps = deltaV_total * 1000;

    fuel_total = m0 * (1 - exp(-deltaV_mps / (Isp * g0)));

    fuel_profile = zeros(size(t));
    idx_burn = t <= burn_duration;


    fuel_profile(idx_burn) = linspace(0, fuel_total, sum(idx_burn));
    fuel_profile(~idx_burn) = fuel_total;


    figure;
    plot(t/3600, fuel_profile, 'r', 'LineWidth', 2);
    xlabel('Time (hours)');
    ylabel('Fuel Used (kg)');
    title('Main Maneuver Fuel Usage');
    grid on;
end
```

References

[1] Vallado, D. A. *Fundamentals of Astrodynamics and Applications*. 4th edition, McGraw-Hill, New York, 2013.

[2] Howard D. Curtis. *Orbital Mechanics for Engineering Students*. 4th edition, Elsevier, Oxford, 2020.